
Conception de circuits numériques

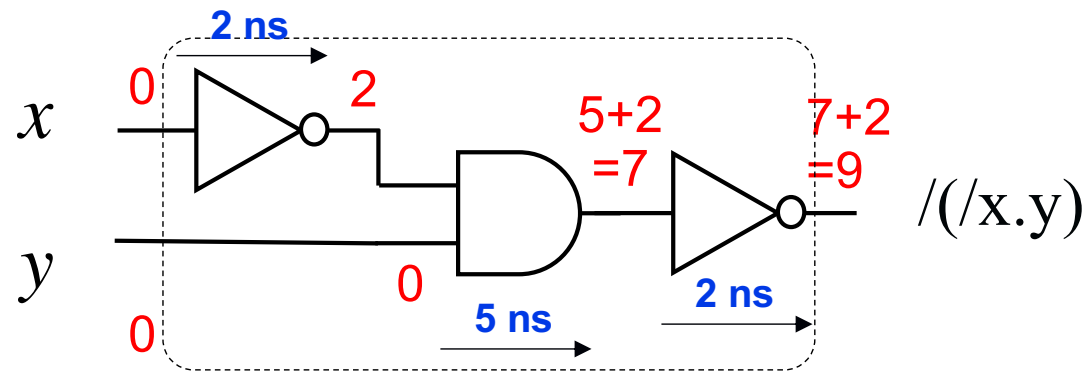
Steven Derrien
Université de Rennes - IRISA

Plan

- 1. Les bases pour (re)démarrer**
2. Conception de circuits pipelinés
3. Chemins de données micro-programmés
4. Intérêt de la Synthèse de Haut Niveau

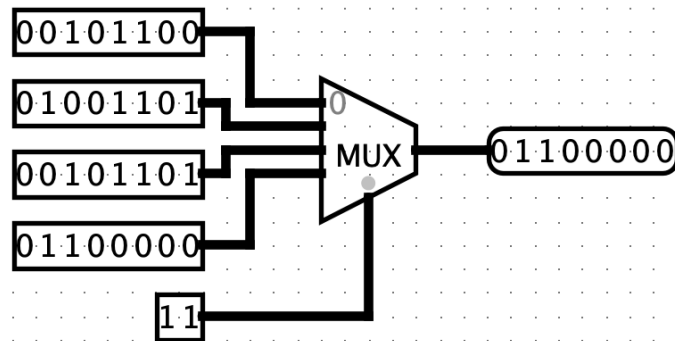
Notion d'assemblage combinatoire

- Assemblage de composants sans *rebouclage*
 - Le graphe formé par l'assemblage est *acyclique*
 - Les sorties de l'assemblage ne dépendent que des entrées
- Délai de propagation d'un assemblage
 - Plus long délai de propagation entre une entrée et une sortie
 - Obtenu par la recherche du **plus long chemin** ds le circuit

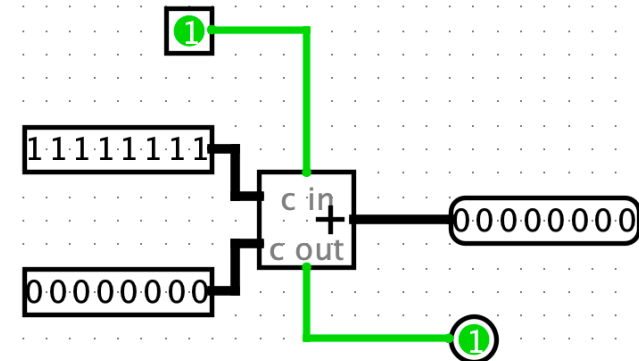


Composant combinatoires usuels

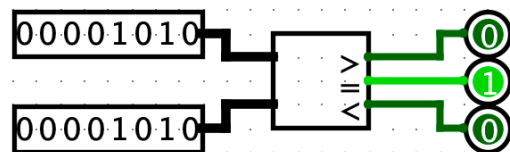
- On utilise également des composants plus complexes



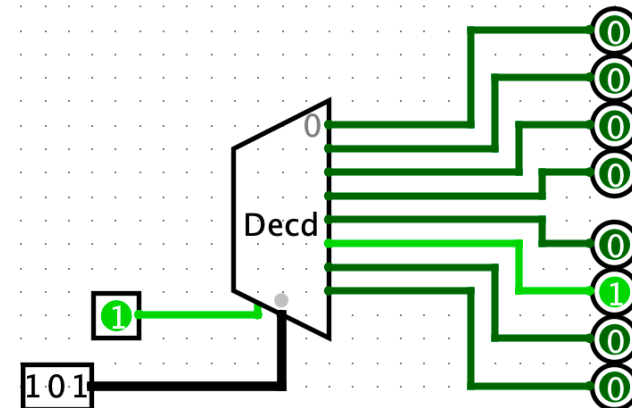
Multiplexeur



Additionneur



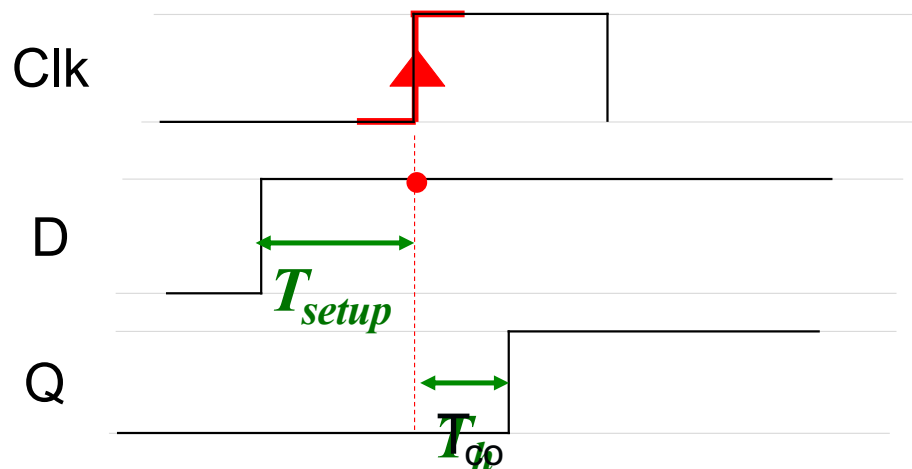
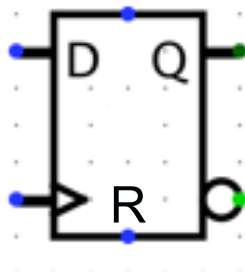
Comparateur



Décodeur

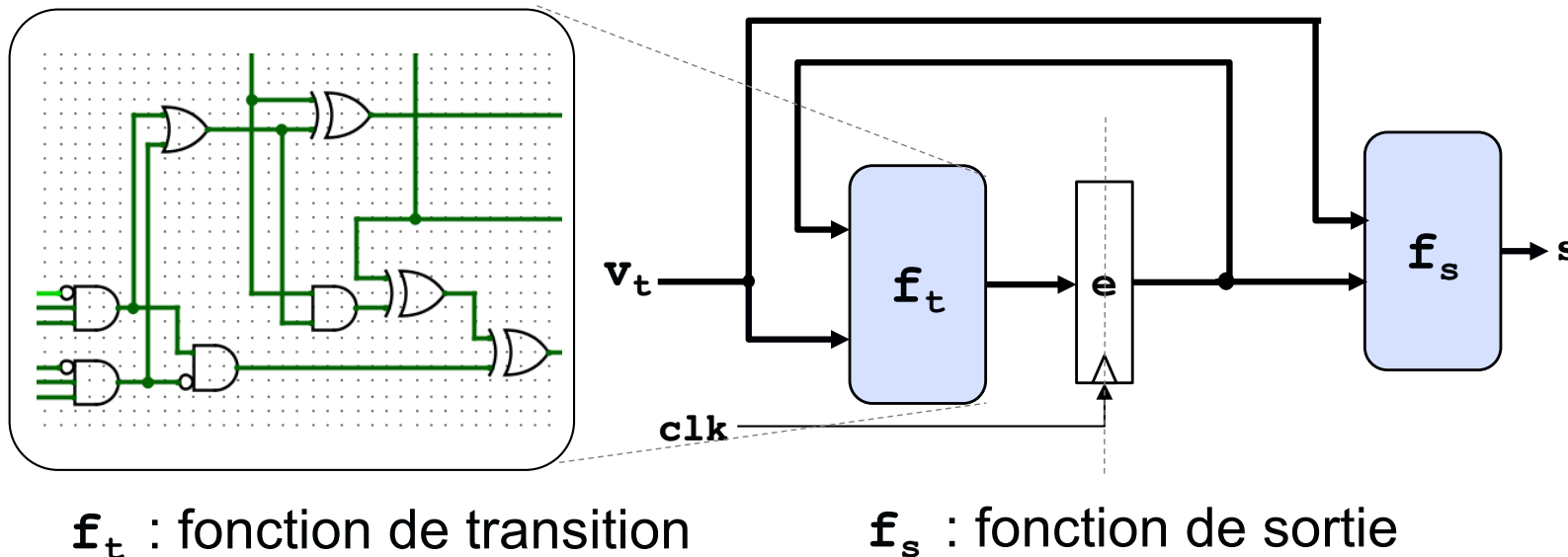
Bascule D sur front

- Mémorise l'entrée D à chaque front montant de **clk**
 - La sortie Q correspond à la valeur mémorisée
 - Utilise une commande de reset asynchrone (R)
- Une bascule D est caractérisée par trois type de délais
 - D doit-être stable au moins T_{setup} avant le front d'horloge
 - Q est mise à jour après un délai max de T_{co}

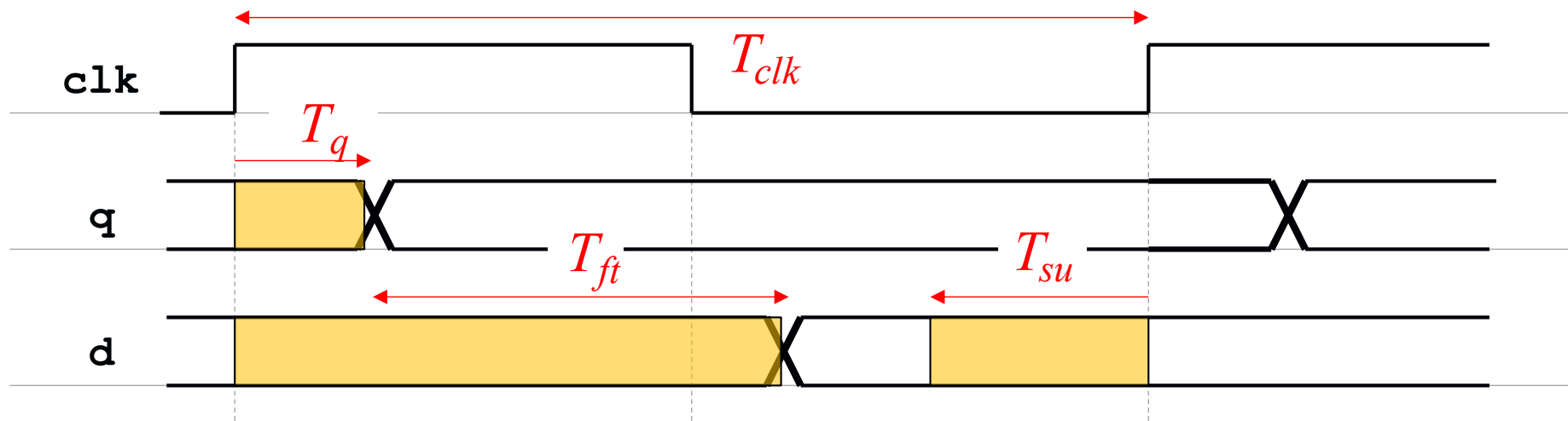
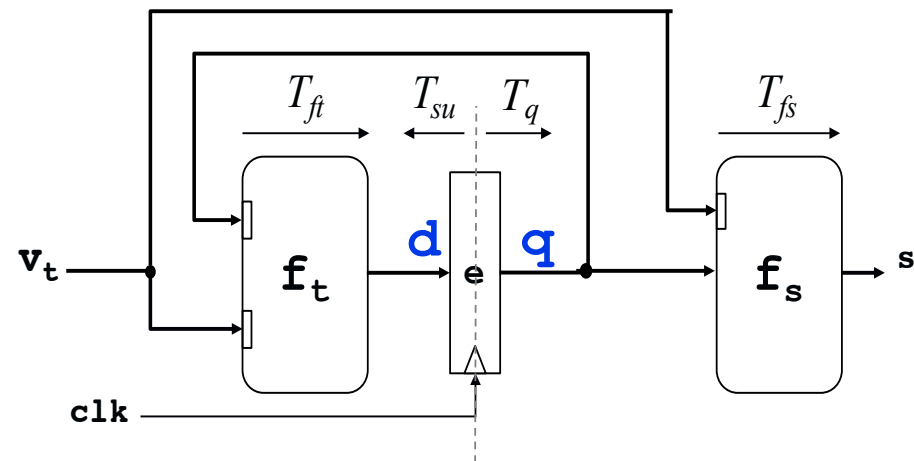


Notion de circuit synchrone

- Circuit séquentiel construit à partir de *D-Flip-Flops*
 - Les D-Flip-Flops partagent **la même horloge et le même reset**
 - On peut avoir des cycles pourvu qu'ils passent par un registre
- Un circuit synchrone se ramène à la forme ci-dessous
 - Les composants f_t , f_s sont des composants combinatoires
 - Le registre e (sur n bits) stocke l'état du circuit.



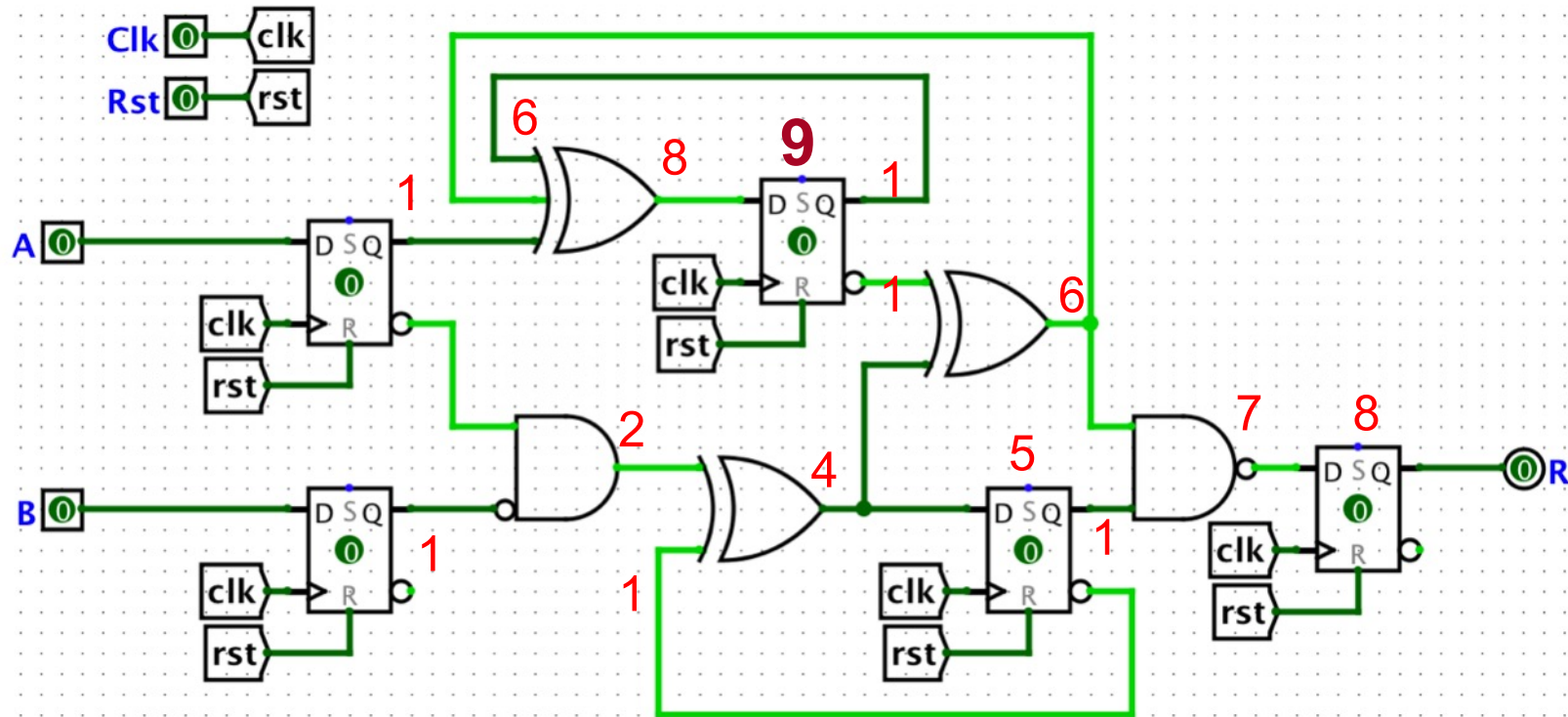
Contraintes temporelles



Pour fonctionner correctement, le circuit doit respecter la contrainte $T_{clk} > T_h + T_{ft} + T_{su}$

Challenge

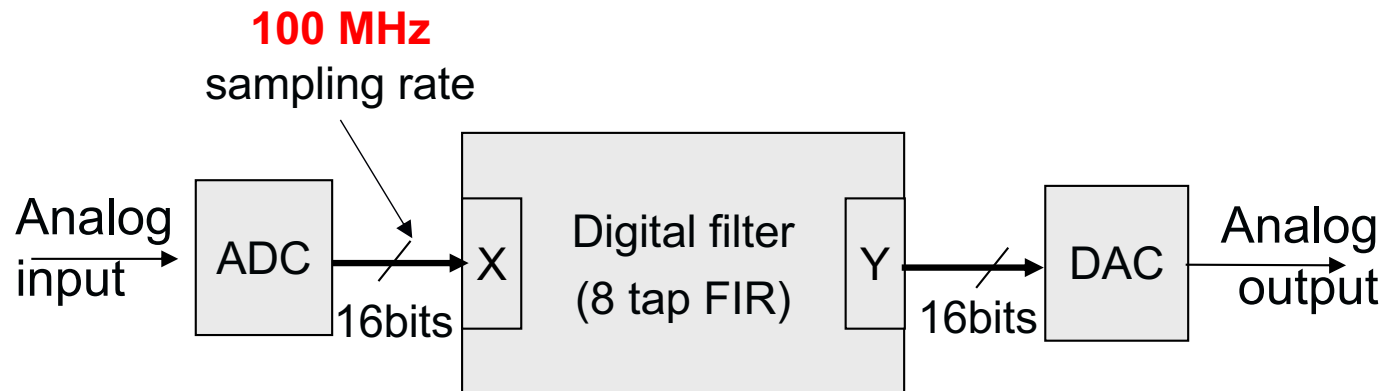
- Quelle période d'horloge minimale dans le cas où
 - On a pour délais $T_{and}=T_{su}=T_h=1ns$ et $T_{xor}=2ns$



Exemple “fil rouge”

- Filtre à réponse impulsionnelle finie à 8 coefficients
 - x, c, et y sont représentés (au minimum) sur 16 bits
 - La cadence d'échantillonnage est imposée à 100MHz

$$y(n) = \sum_{i=0}^7 x(n-i) \cdot c(i)$$

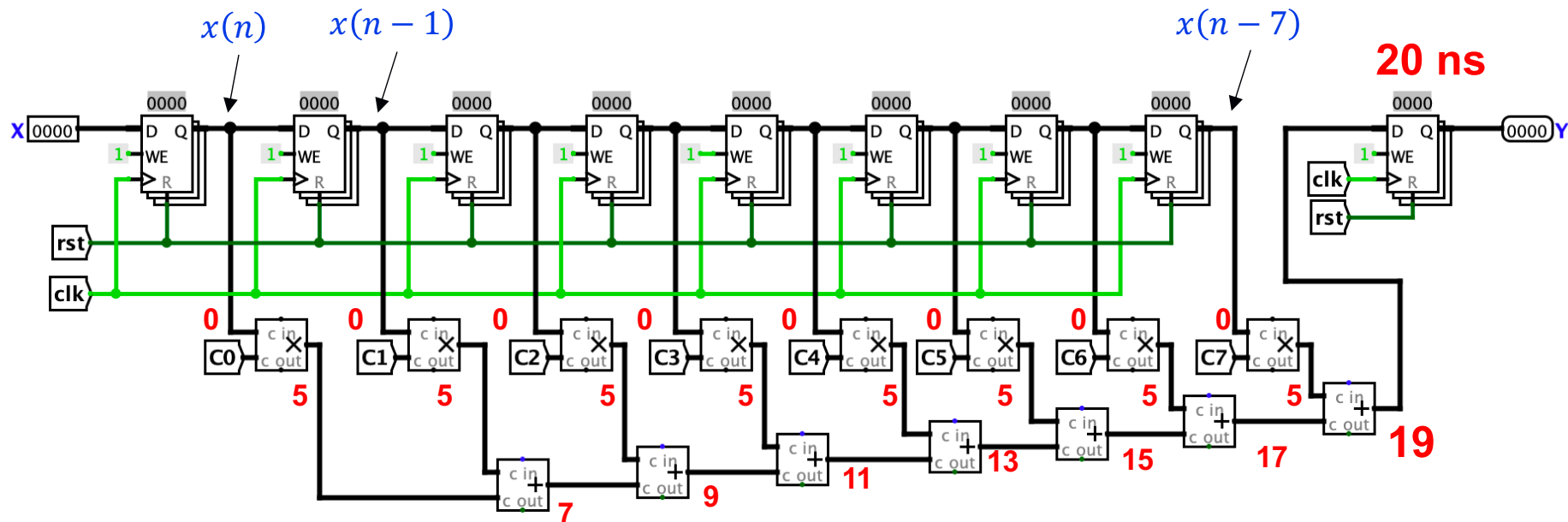


- Comment réaliser un circuit pour ce traitement ?

Solution 1

- Somme de produit réalisée de manière combinatoire
 - 8 multiplications + 8 addition en “parallèle” à chaque cycle

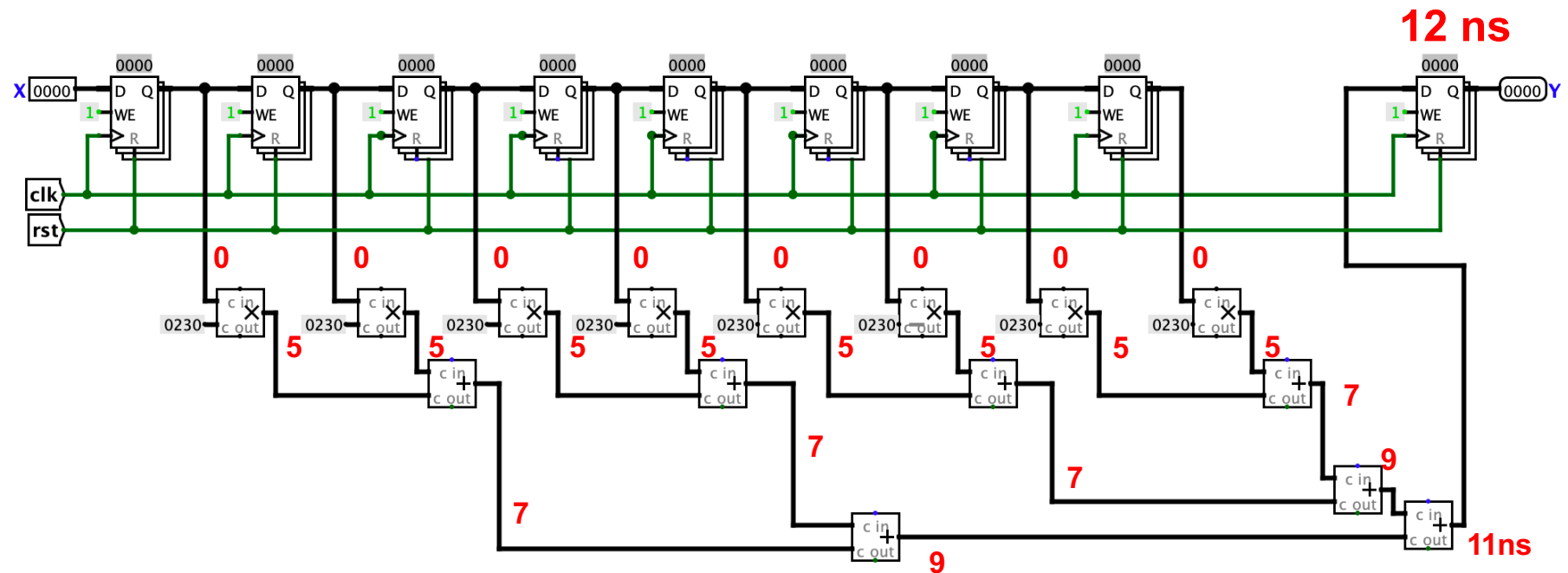
$$y(n) = \sum_{i=0}^7 x(n-i) \cdot c(i)$$



Ne satisfait pas la contrainte de performance ! $T_{\min} = 20\text{ns}$
 $f_{\max} = 50\text{ MHz}$

Solution 2

- On exploite en plus l'associativité de l'addition entière
- Utilisation d'un arbre d'additionneurs binaires



Mieux mais toujours pas suffisant !

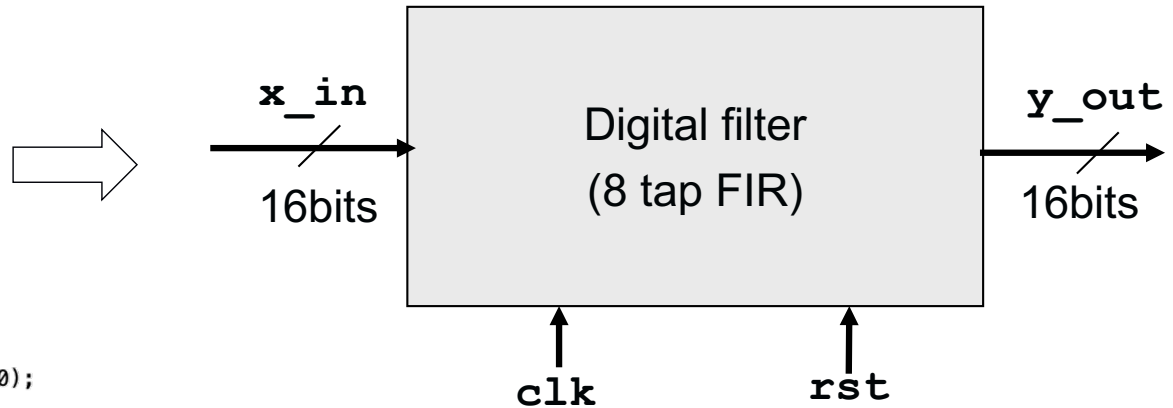
$T_{\min} = 12\text{ ns}$
 $F_{\max} = 83\text{ MHz}$

Spécification en VHDL

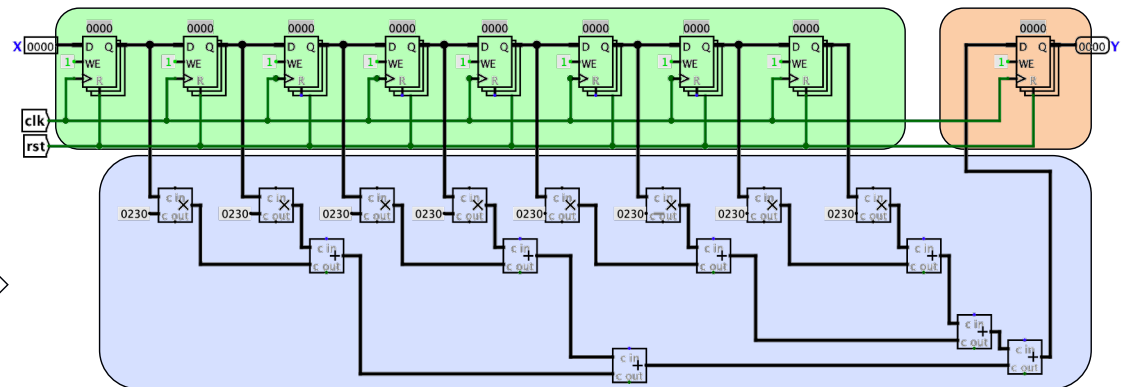
```

8  library IEEE;
9  use IEEE.std_logic_1164.all;
10 use IEEE.numeric_std.all;
11
12 entity fir is
13 port (
14     clk : IN STD_LOGIC;
15     rst_n : IN STD_LOGIC;
16     x_in : in STD_LOGIC_VECTOR (7 downto 0);
17     y_out : OUT STD_LOGIC_VECTOR (7 downto 0)
18 );
19 end fir;
20
21
22 architecture RTL of fir is
23
24     subtype int16 is STD_LOGIC_VECTOR (15 downto 0);
25     type array16x16 is array (0 to 7) of int16;
26     signal x,c : array16x16;
27
28 begin
29
30     process (clk)
31         variable sum : std_logic_vector(47 downto 0);
32     begin
33         if rst='1' then
34             y_out <= (others=>'0');
35         elsif rising_edge(clk) then
36
37             x(0) <= x_in;
38
39             for i in 1 to 7 loop
40                 x(i) <= x(i-1);
41             end loop; -- i
42
43             sum := 0;
44             for i in 0 to 7 loop
45                 sum := sum + c(i)*x(i);
46             end loop; -- i
47
48             yout <= sum;
49
50         end if;
51     end process;
52
53 end RTL;
54

```



L'outil de synthèse RTL optimisera les additions automatiquement

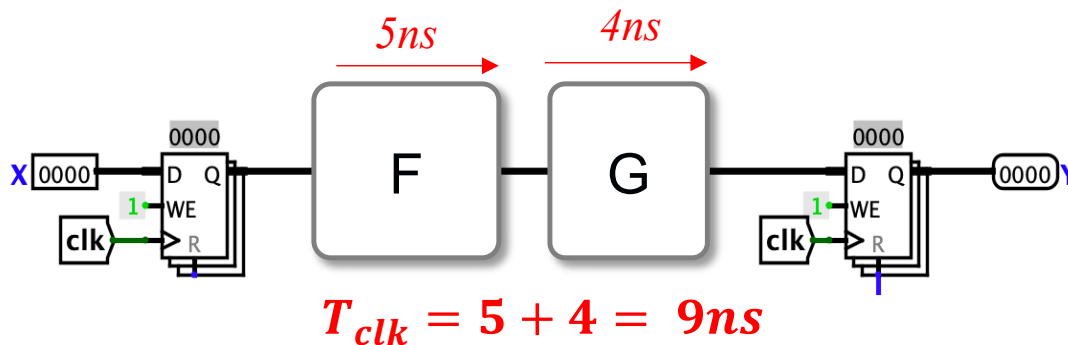


Plan

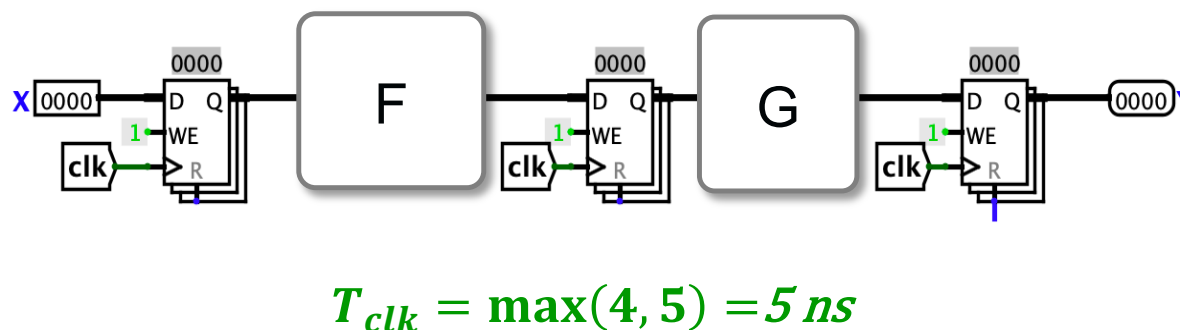
1. Les bases pour (re)démarrer
- 2. Conception de circuits pipelinés**
3. Chemins de données micro-programmés
4. Intérêt de la Synthèse de Haut Niveau

Pipeline

- Comment réduire le chemin critique d'un circuit ?
 - On peut insérer des registres pour casser les chemins longs



$$y(n) = G(F(x(n-2)))$$

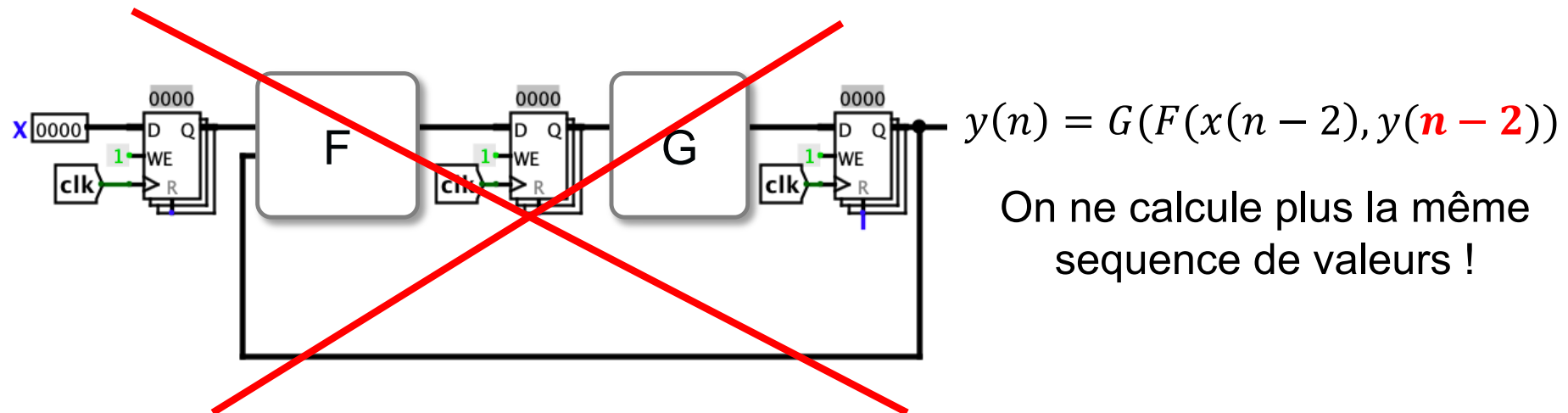
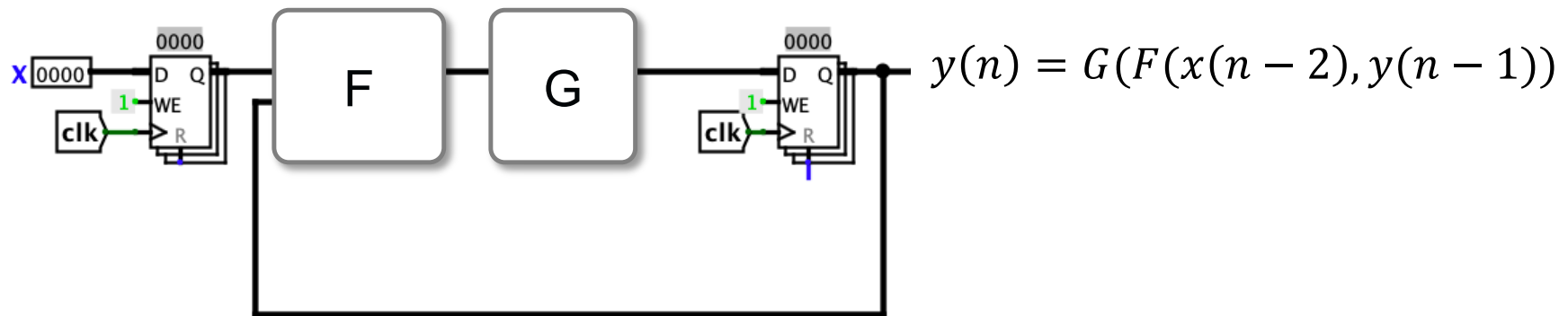


$$y(n+1) = G(F(x(n-2)))$$

La même sequence mais
avec un retard de 1 cycle sur
la sortie

Pipeline bis

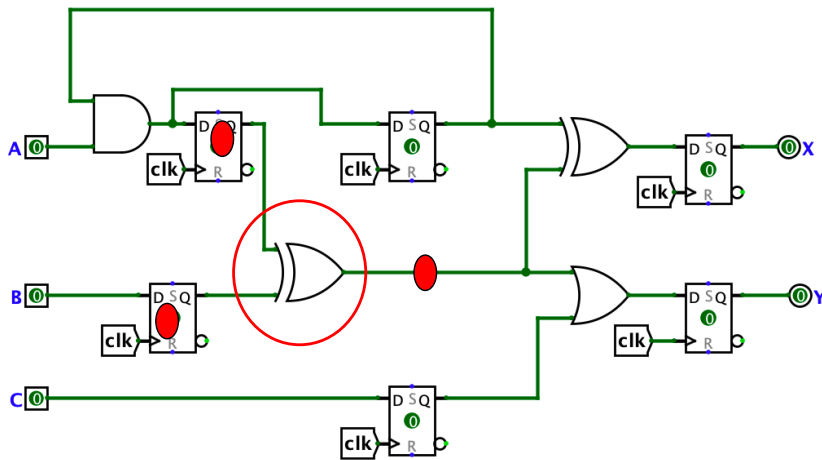
- Comment pipeliner le circuit ci-dessous ?



Comment savoir si une transformation de circuit est correcte ?

Transformation de *retiming*

- Déplacer des registres sans changer le calcul réalisé
 - Utilisé pour le pipeline quand il y a des registres « mal placés »
- Principe :
 - Enlève/Ajoute 1 registre sur tous les arcs entrants du bloc
 - Ajoute/Enlève 1 registre sur tous les arcs entrants du bloc

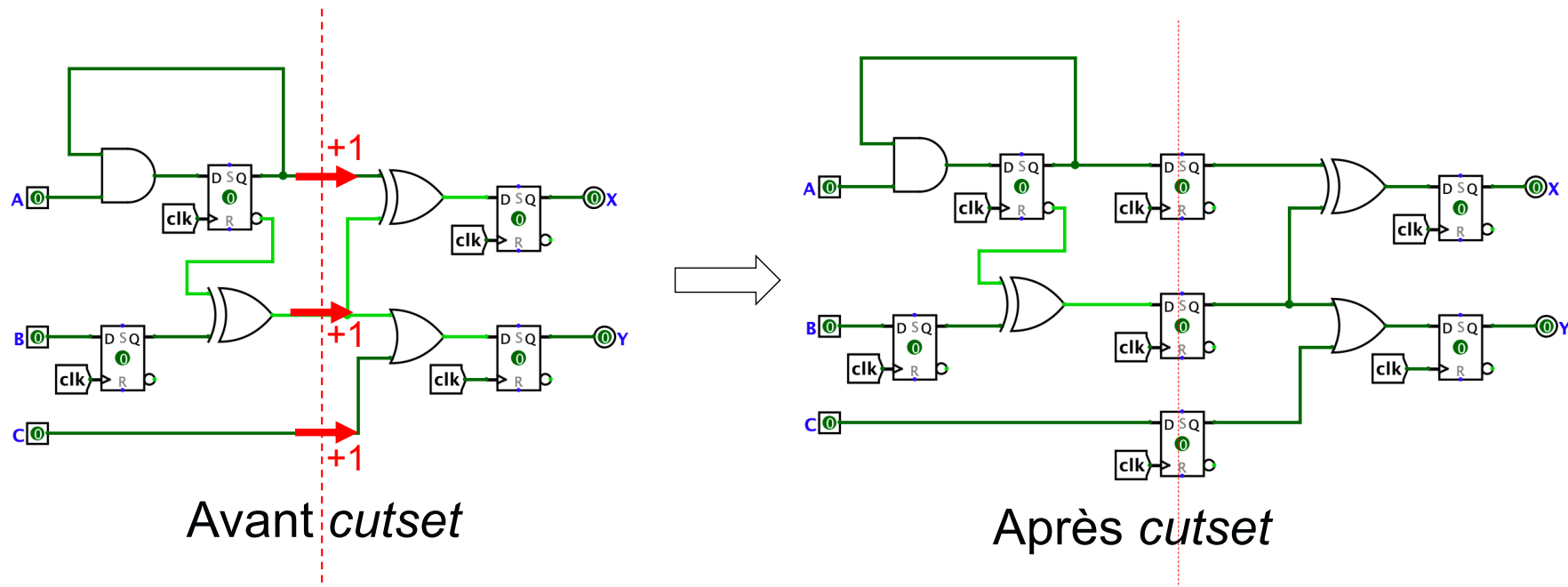


Avant retiming

Et si il n'y a pas de registres à déplacer ?

Le cut-set retiming [1]

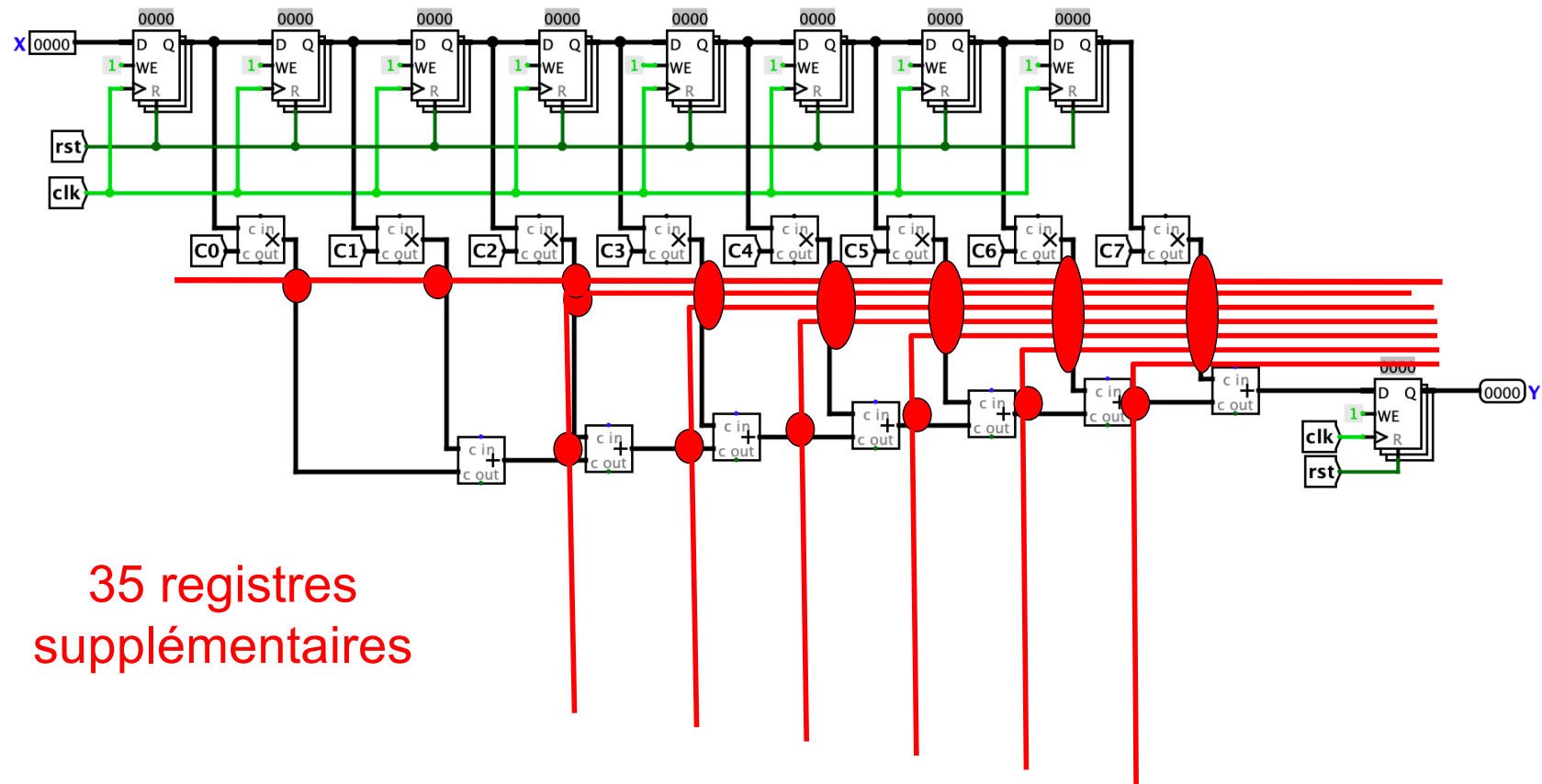
- Transformation qui ajoute des registres sur un chemin
 - Permet de préserver la fonctionnalité du circuit
 - Basé sur la notion de **coupe** sur le graphe du circuit
- Qd le flot de données traverse la coupe ds un seul sens
 - Alors on peut ajouter n registres sur chaque intersections



[1] Leiserson & Saxe, *Retiming synchronous circuits*, Algorithmica, 1986

Challenge

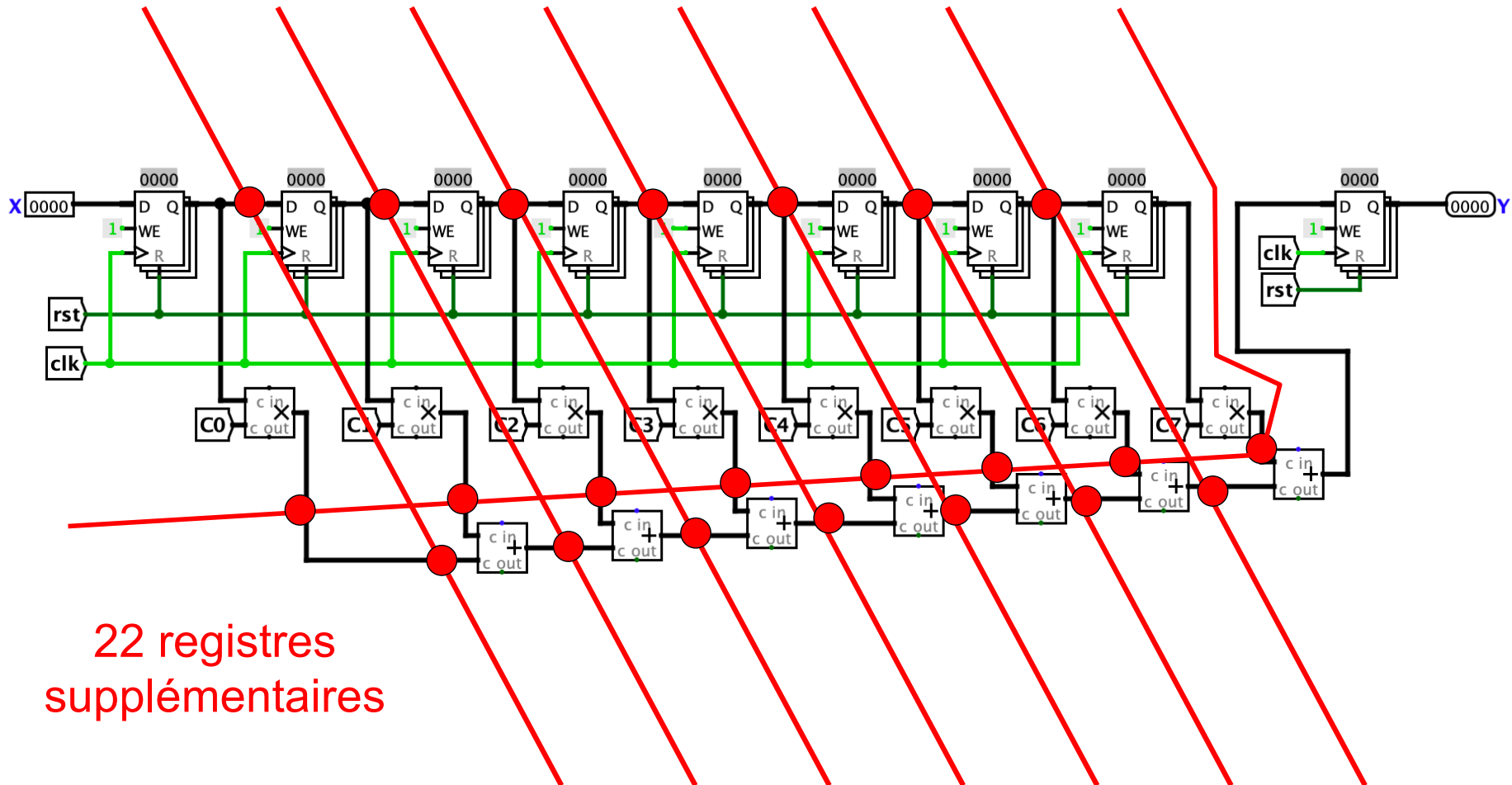
- Comment pipeliner le circuit ci-dessous via des coupes ?



35 registres
supplémentaires

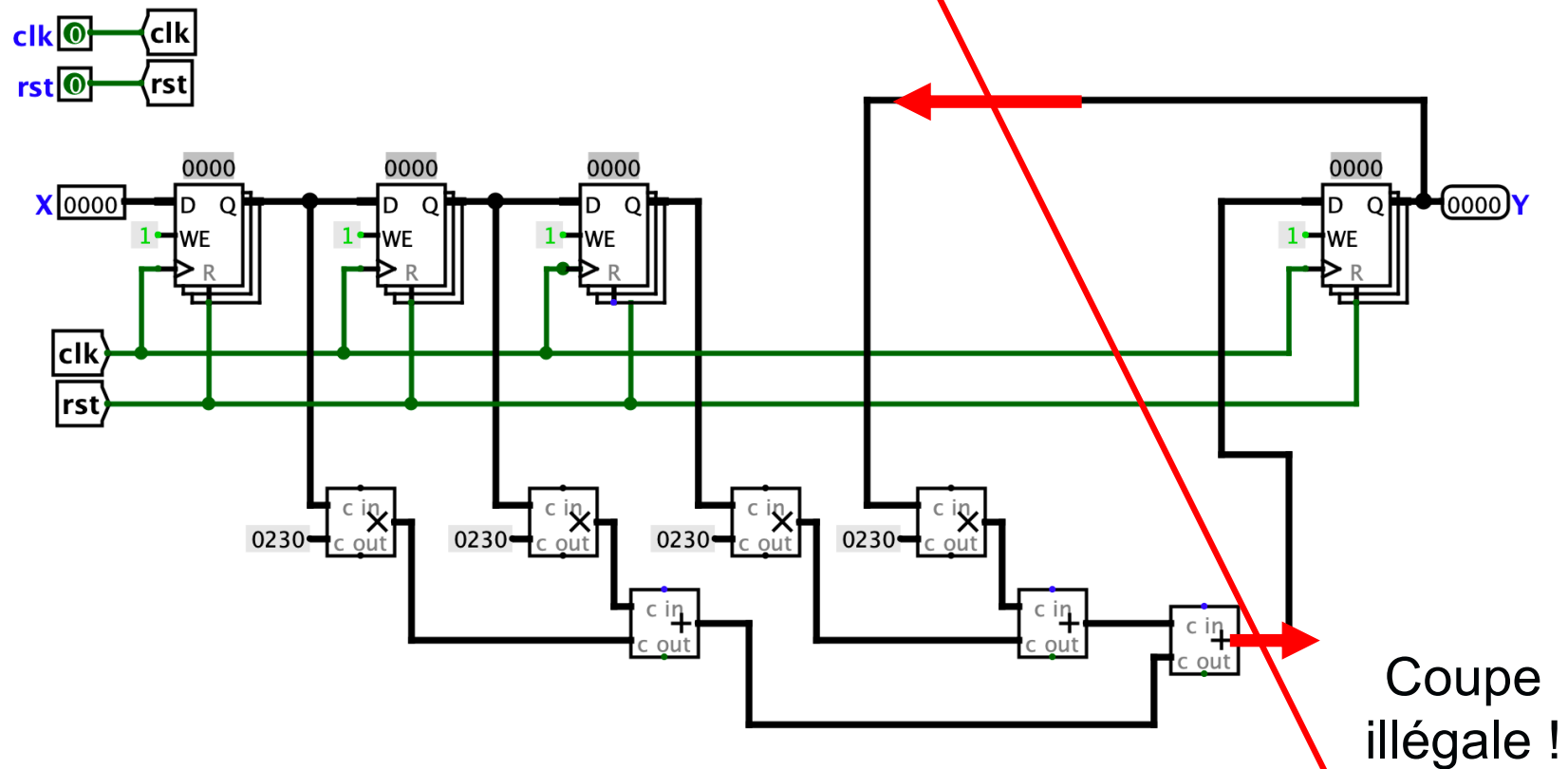
Challenge

- Comment pipeliner le circuit ci-dessous via des coupes ?



Challenge

- Peut-on pipeliner le circuit ci-dessous avec des coupes ?



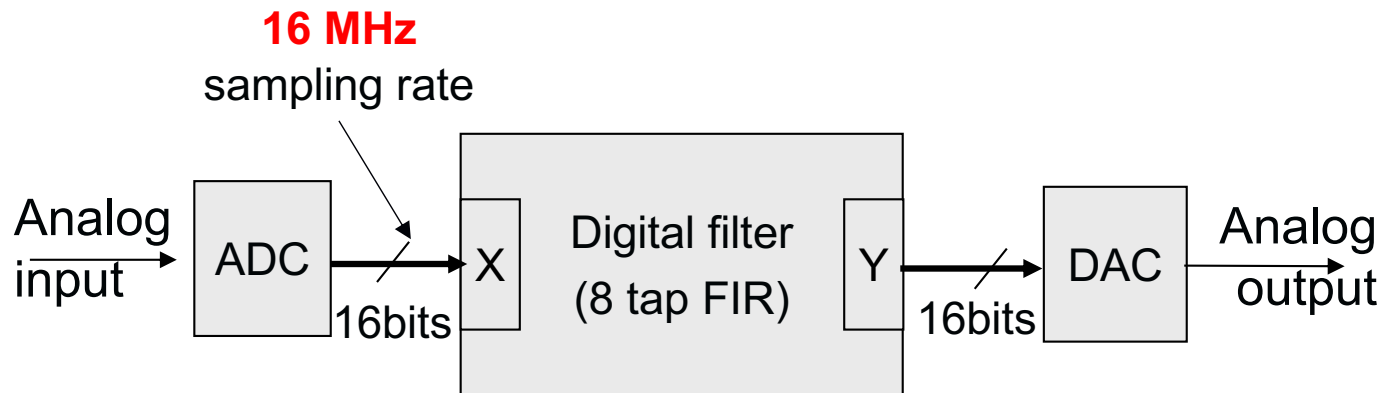
Plan

1. Les bases pour (re)démarrer
2. Conception de circuits pipelinés
- 3. Chemins de données micro-programmés**
4. Intérêt de la Synthèse de Haut Niveau

Exemple “fil rouge” bis

- Cette fois-ci la contrainte de performance est différente
 - La fréquence d'échantillonnage est 16MHz au lieu de 100 MHz

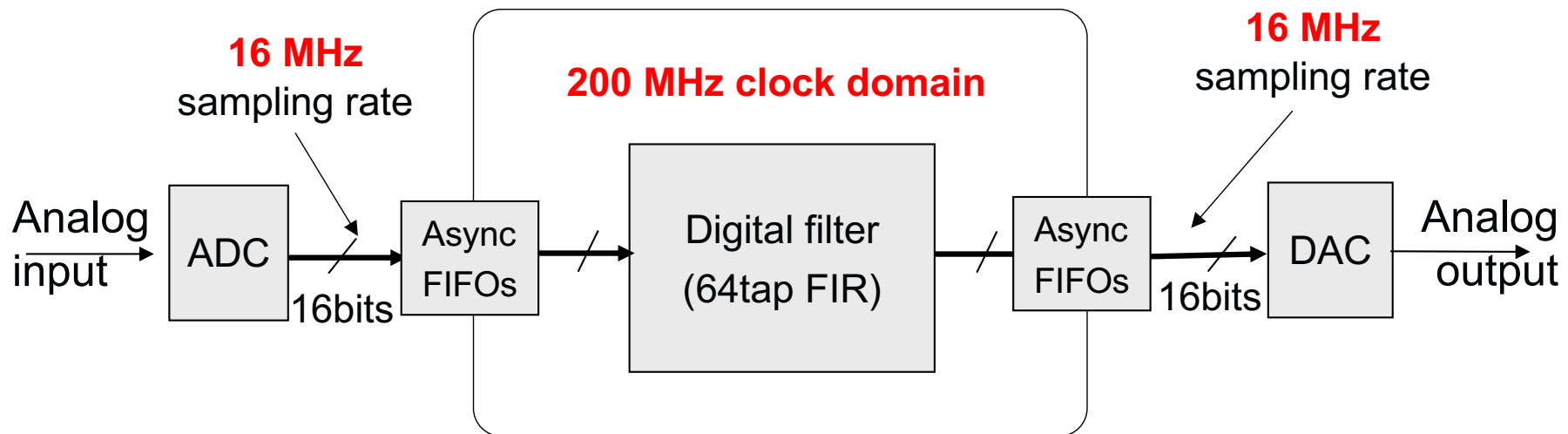
$$y(n) = \sum_{i=0}^7 x(n-i) \cdot c(i)$$



- Peut-on éviter d'utiliser 8 multiplieurs pour le circuit ?

Découpler f_{sample} et f_{clk}

- Utilise une horloge distincte pour le FIR ($f_{clk} \gg f_{sample}$)
 - On choisit $f_{clk}=200MHz$ for $f_{sample}=16 MHz$ ($\sim 12,5x$)
 - Connexions via des FIFO asynchrones au domaine f_{sample}

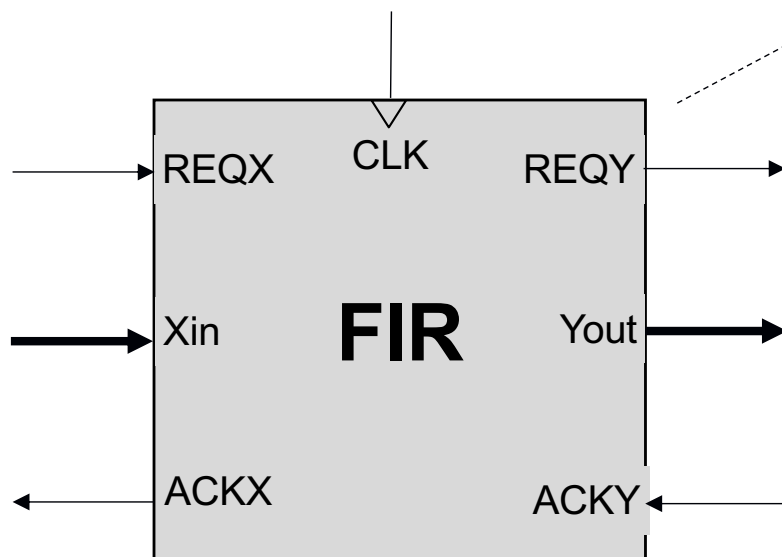


- Le composant doit produire une sortie tous les 12,5 cycles

Approche micro-programmée

- Le composant va réaliser les calculs séquentiellement

$$y(n) = \sum_{i=0}^7 x(n-i) \cdot c(i)$$

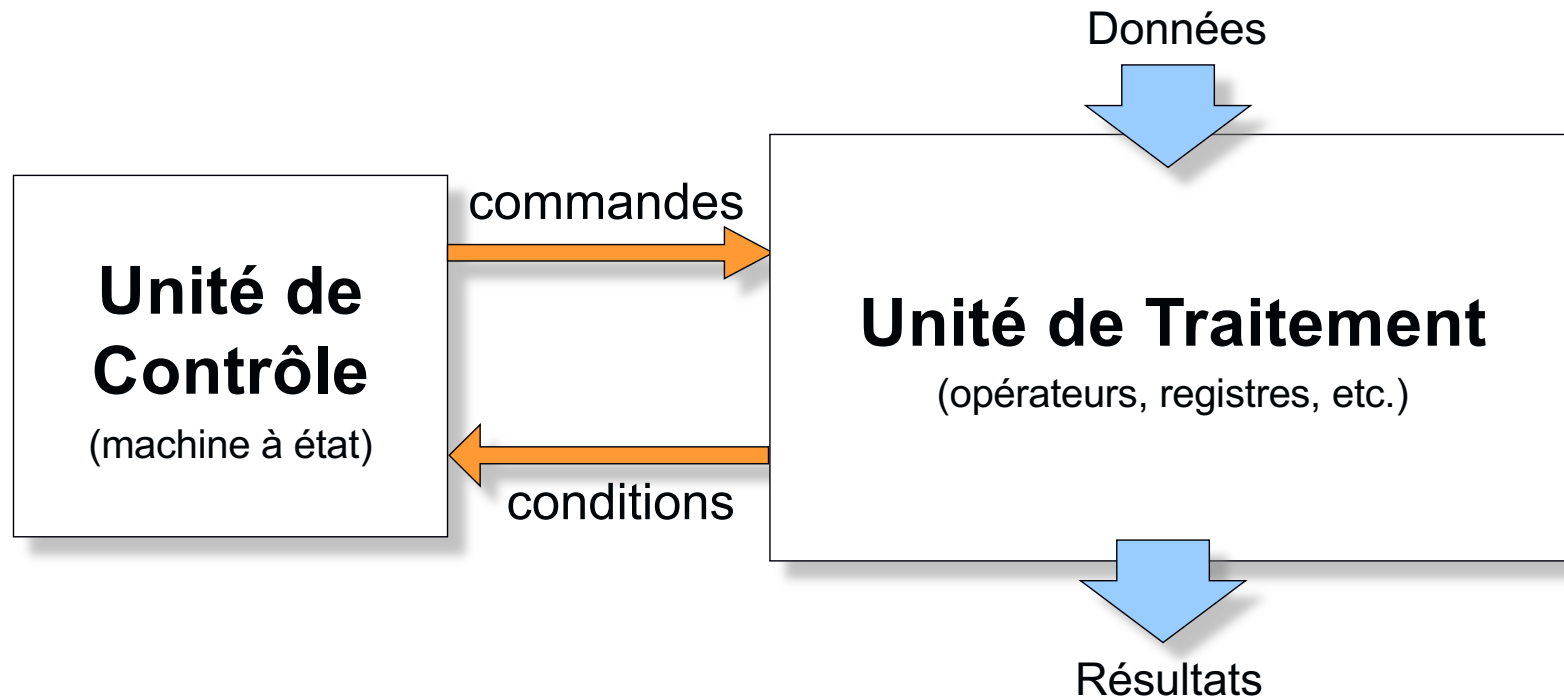


```
while(1) {  
    x[k] = read_Xin();  
    acc = 0;  
    for(i=0; i<8; i++) {  
        prod = x[k-i] * C[i];  
        acc = acc + prod  
        k= (k+1)%8;  
    }  
    k= (k+1)%8;  
    write_Yout(acc);  
}
```

Tampon circulaire pour **x[8]**

Décomposition UT/UC

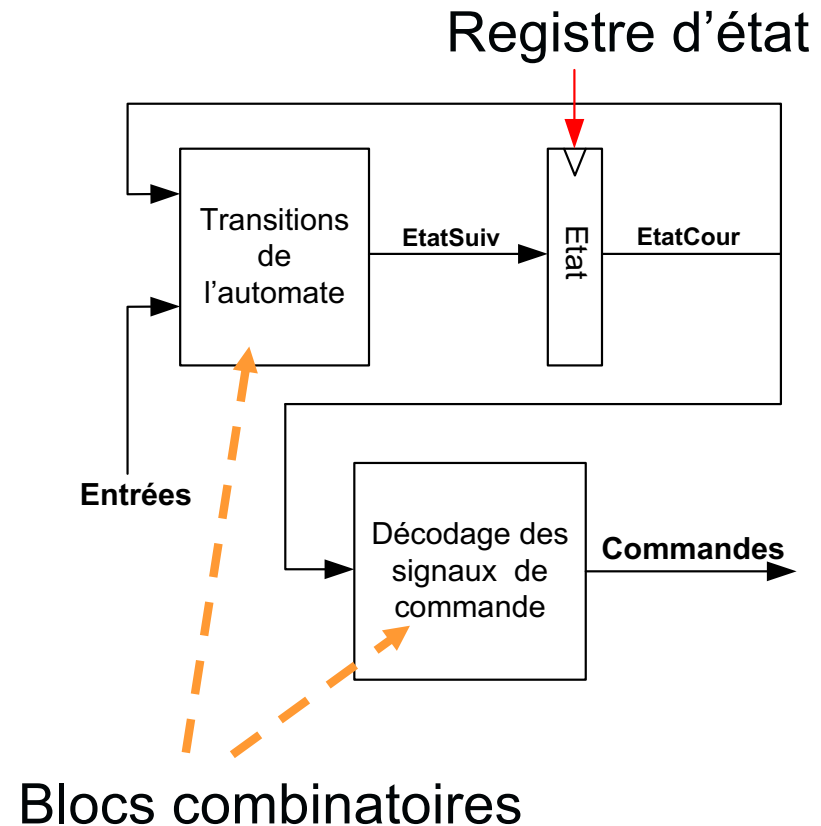
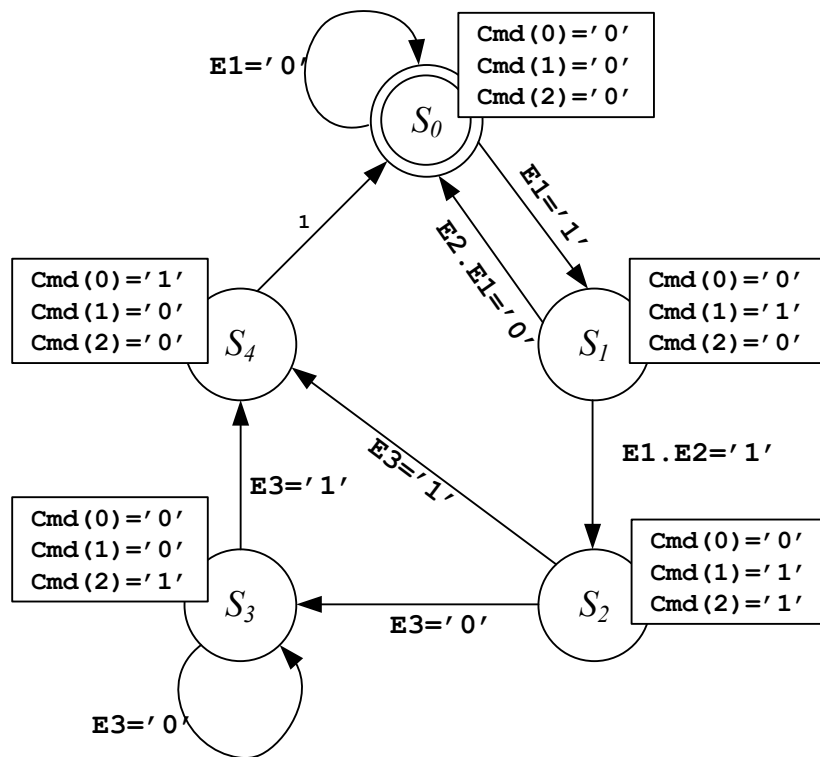
- Traitements complexes → décomposition UC + UT



Commandes	Générées par l'UC Définissent les opérations effectuées dans le cycle
Conditions	Issues de l'UT et testées par l'UC Influencent le séquençement de l'UC

Unité Contrôle = machine à état de Moore

- Moore : commandes dépendent **uniquement** de l'état



Hand-shake

- Il faut un protocole pour lire/écrire depuis/vers ls FIFOs

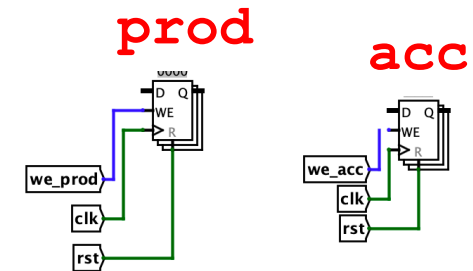
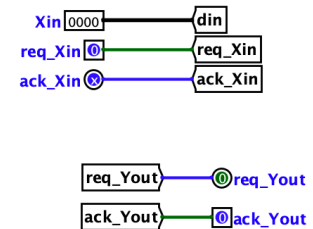
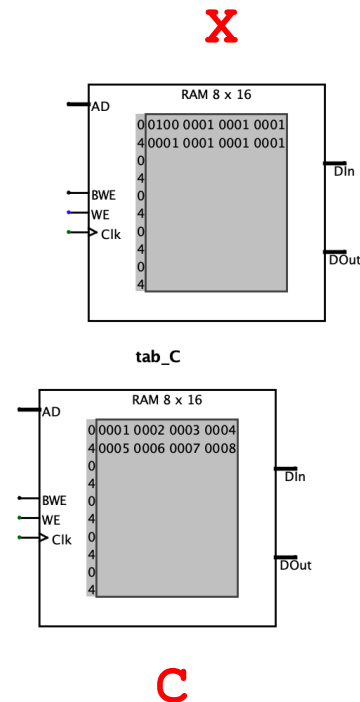
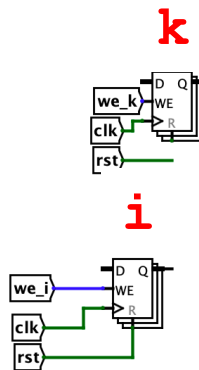


- ① La FIFO indique un donné disponible ($x_{rdy}=1$)
- ② Le FIR consomme la donnée et place ($x_{rd}=1$),
- ③ Le FIR calcule $y(n)$ à partir des $x(n-k)$ et attend $y_{rdy}=1$
- ④ Le FIR émet la donnée en sortie et place ($y_{wr}=1$)

1. Ajout des registres/mémoire

- Associés à chaque variable du programme

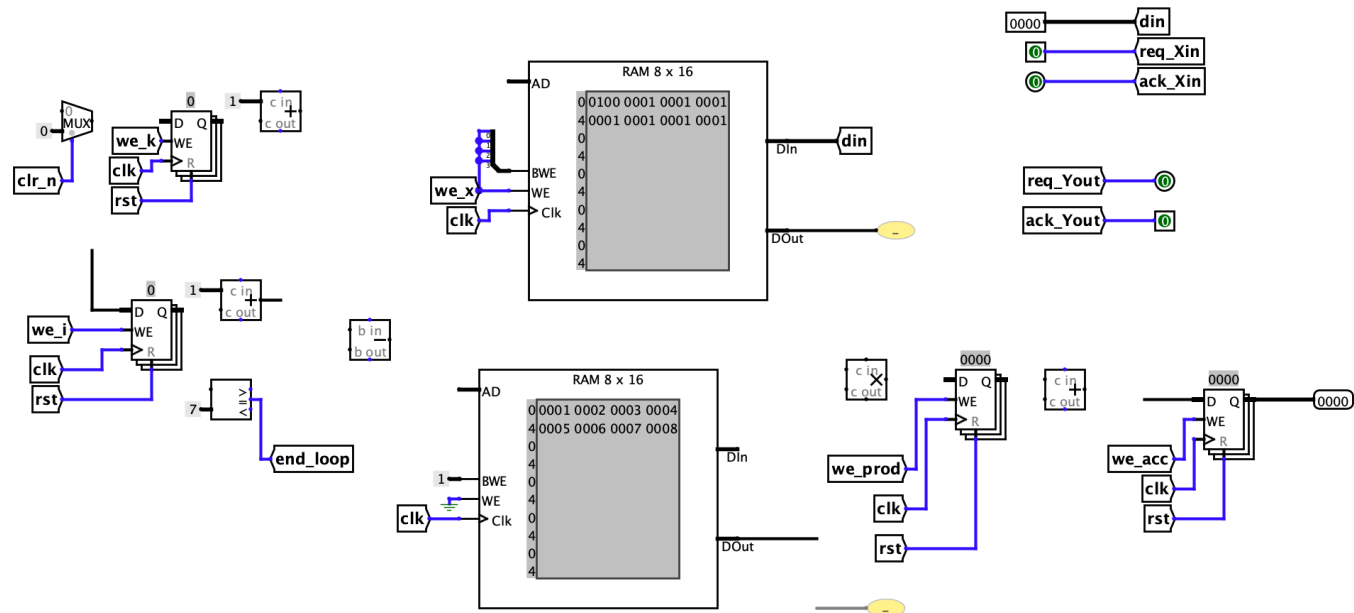
```
while(1) {
  x[k] = read_Xin();
  acc = 0;
  for(i=0; i<8; i++) {
    prod = x[k-i]*C[i];
    acc = acc + prod;
    k = (k+1)%8;
  }
  k = (k+1)%8;
  write_Yout(acc);
}
```



2. Ajout des opérateurs

- En fonction des opérations utilisées dans l'algorithme
 - Possibilité de parallélisme au niveau opérateur

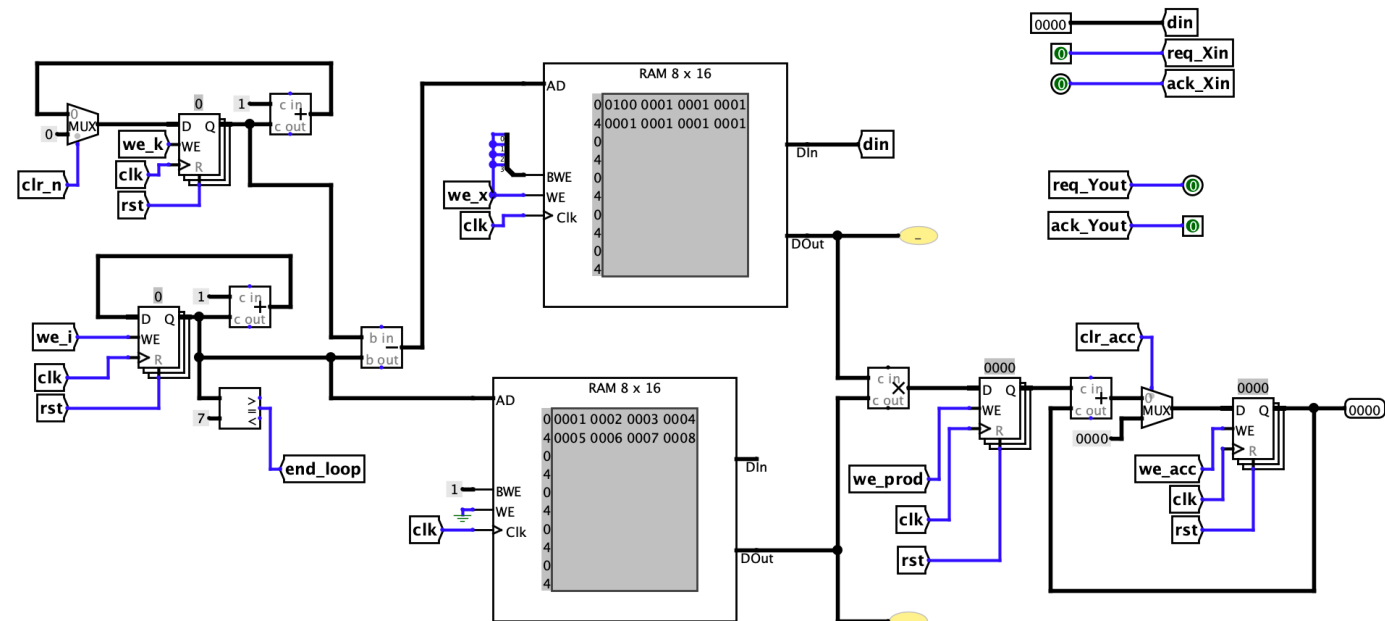
```
while(1) {
    x[k] = read_Xin();
    acc = 0;
    for(i=0;i<8;i++) {
        prod = x[k-i] * C[i];
        acc = acc + prod;
        k = (k+1)%8;
    }
    k = (k+1)%8;
    write_Yout(acc);
}
```



3. Ajout des chemins de données

- En fonction des affectations et opérandes des calculs
 - Nécessite souvent des multiplexeurs

```
while(1) {
    x[k] = read_Xin();
    acc = 0;
    for(i=0;i<8;i++) {
        prod = x[k-i]*C[i];
        acc = acc + prod;
        k= (k+1)%8;
    }
    k= (k+1)%8;
    write_Yout(acc);
}
```



command	effect
we_i	$i = i + 1$
we_k	$k = (k + 1) \% 8$
we_prod	$mul = x_q * c_q$

command	effect
we_acc./clr_acc	$acc = acc + mul$
we_acc.clear	$acc = 0$
we_x	$X[i] = x_din$

4. Construction du contrôle

- Séquencement naïf des opérations (pas de parallélisme)
 - Pour simplifier on ignore la gestion des FIFOs

command	effect
we_i	i= i+1
we_k	k=(k+1)%8
we_prod	mul=x_dout*c_dout

command	effect
we_acc./clr_acc	acc=acc+mul
we_acc.clear	acc=0
we_x	X[i]=x_din

```
while(1) {  
    x[k] = read_Xin();  
    acc =0;  
    for(i=0;i<8;i++) {  
        prod = x[k-i]*C[i];  
        acc = acc + prod  
        k= (k+1)%8;  
    }  
    k= (k+1)%8;  
    write_Yout(acc);  
}
```

command	effect
we_x	X[i]=x_din
we_acc.clear	acc=0
-	x_q = X[k]
we_prod	mul=x_q*c_q
we_acc	acc=acc+mul
we_k,we_i	k=(k+1)%8, i++, i<8
We_k	k=(k+1)%8

x8

35 cycles par échantillon

4. Construction du contrôle

- Séquencement avec parallélisme entre opérations
 - Mais limité par les dépendances entre calculs

command	effect
we_i	i= i+1
we_k	k=(k+1)%8
we_prod	mul=x_dout*c_dout

command	effect
we_acc./clr_acc	acc=acc+mul
we_acc.clear	acc=0
we_x	X[i]=x_din

```

while(1) {
  x[k] = read_Xin();
  acc =0;
  for(i=0;i<8;i++) {
    prod = x[k-i]*C[i];
    acc = acc + prod
    k= (k+1)%8;
  }
  k= (k+1)%8;
  write_Yout(acc);
}

```

command	effet
we_x, we_acc.clear	X[i]=x_din, acc=0
-	x_q = X[k], k=(k+1)%8
we_prod	mul=x_q*c_q
we_acc, we_k	acc=mul+acc, i++, i<8
we_k	k=(k+1)%8

x8

26 cycles par échantillon

4. Construction du contrôle

- Séquencement avec recouvrement (pipeline de boucle)
 - On recouvre l'exécution de trois itérations de la boucle

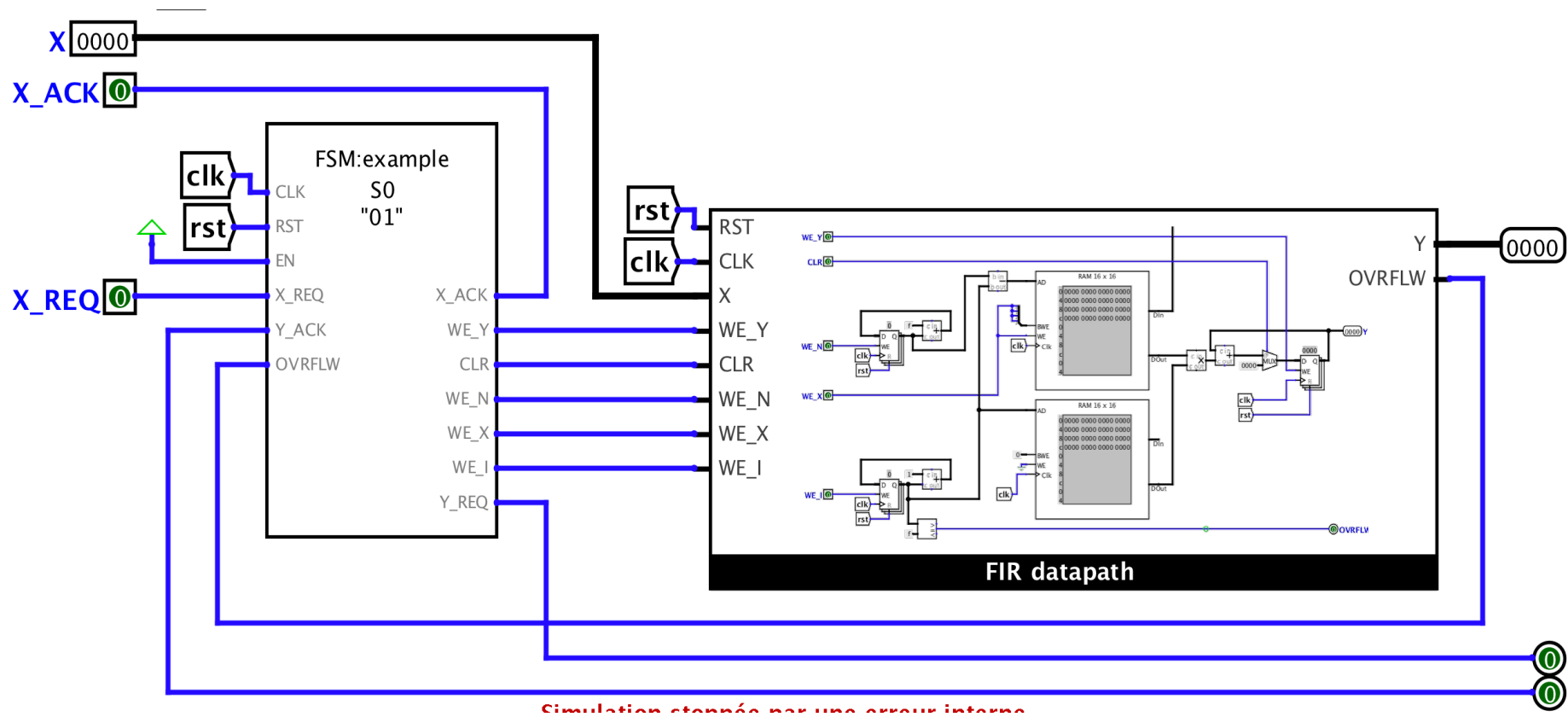
```
while(1) {
    x[k] = read_Xin();
    acc = 0;
    for(i=0;i<8;i++) {
        prod = x[k-i]*C[i];
        acc = acc + prod;
        k = (k+1)%8;
    }
    k = (k+1)%8;
    write_Yout(acc);
}
```

	operations			
1	X[i]=x_din			acc=0
2	x_q = X[k]	k=(k+1)%8, i++		
3	x_q = X[k]	k=(k+1)%8, i++	prod= x_q*c_q	
4	x_q = X[k]	k=(k+1)%8, i++	prod= x_q*c_q	acc=acc+mul
5	x_q = X[k]	k=(k+1)%8, i++	prod= x_q*c_q	acc=acc+mul
6	x_q = X[k]	k=(k+1)%8, i++	prod= x_q*c_q	acc=acc+mul
7	x_q = X[k]	k=(k+1)%8, i++	prod= x_q*c_q	acc=acc+mul
8	x_q = X[k]	k=(k+1)%8, i++	prod= x_q*c_q	acc=acc+mul
9	x_q = X[k]	k=(k+1)%8, i++	prod= x_q*c_q	acc=acc+mul
10			prod= x_q*c_q	acc=acc+mul
11	k=(k+1)%8			acc=acc+mul

11 cycles par échantillon (12,5 attendus)

Vue d'ensemble

■ Machine à état + Unité de traitement



700 LOC de VHDL, combien de temps pour la conception, validation ?

Ce n'est pas fini ...

- Après avoir écrit le modèle VHDL, il faut ensuite
 - Faire la synthèse RTL/Logique
 - Modifier le VHDL
 - Simuler le VHDL

- On souhaite faire tourner le circuit à 200Mhz
 - Que faire si la synthèse donne $f_{clk}=183$ Mhz malgré nos efforts
 - Il faut extraire plus de parallélisme → recommencer à zéro !

Idéalement on souhaiterait pouvoir facilement essayer plusieurs implémentations pour trouver celle qui est la plus pertinente.

Faire cette exploration au niveau RTL va prendre trop de temps

Plan

1. Les bases pour (re)démarrer
2. Conception de circuits pipelinés
3. Chemins de données micro-programmés
4. **Intérêt de la Synthèse de Haut Niveau**

Discussion

- What about the complexity of the VHDL code ?
- How many VHLD components, lines of code?
- How does it compare with 1st use case ?
- Is it likely to be easier of harder to debug ?
- How long do you think you would need to make it work ?